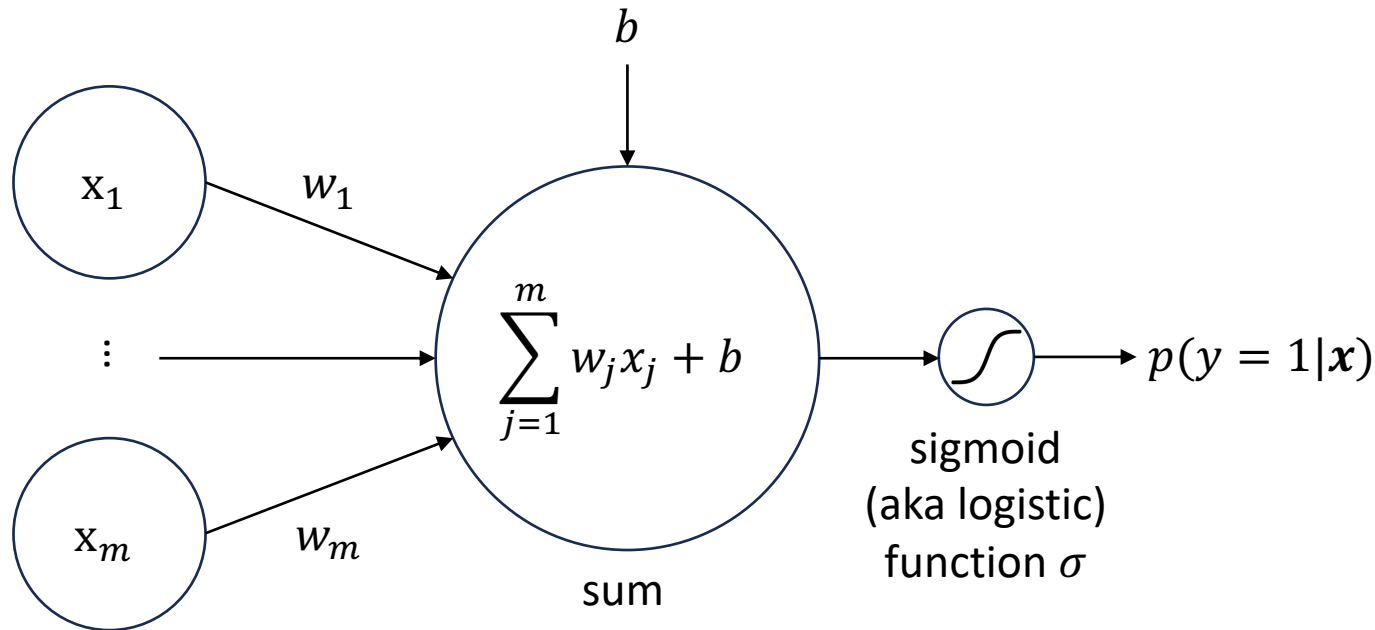


Shallow and deep neural networks

Matt Engelhard

Today: how can we modify logistic regression to learn complex, nonlinear relationships?

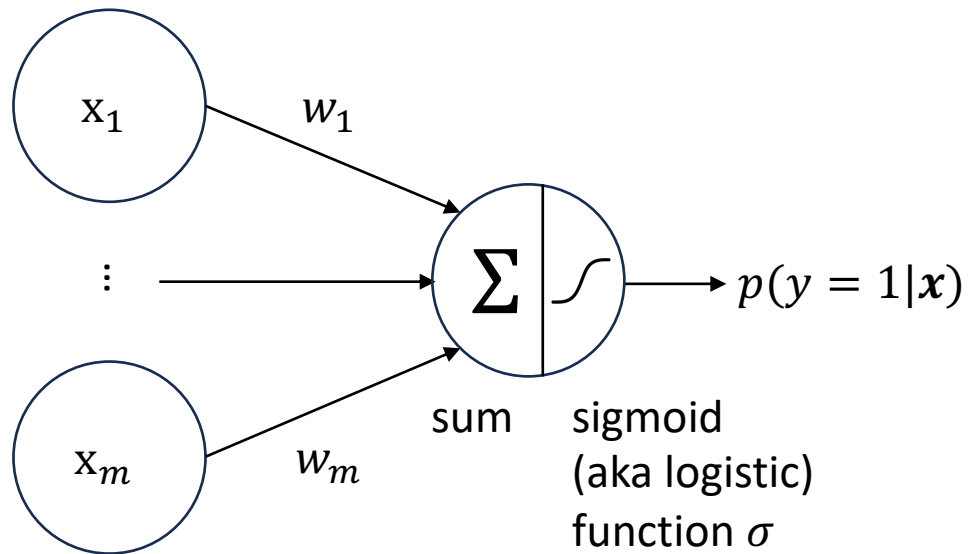


$$p(y = 1|\mathbf{x}) = \sigma\left(\sum_{j=1}^m w_j x_j + b\right)$$

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Let's make our notation more concise



We will later use h to denote the outputs of our probabilistic perceptrons (i.e., neurons) when building neural networks

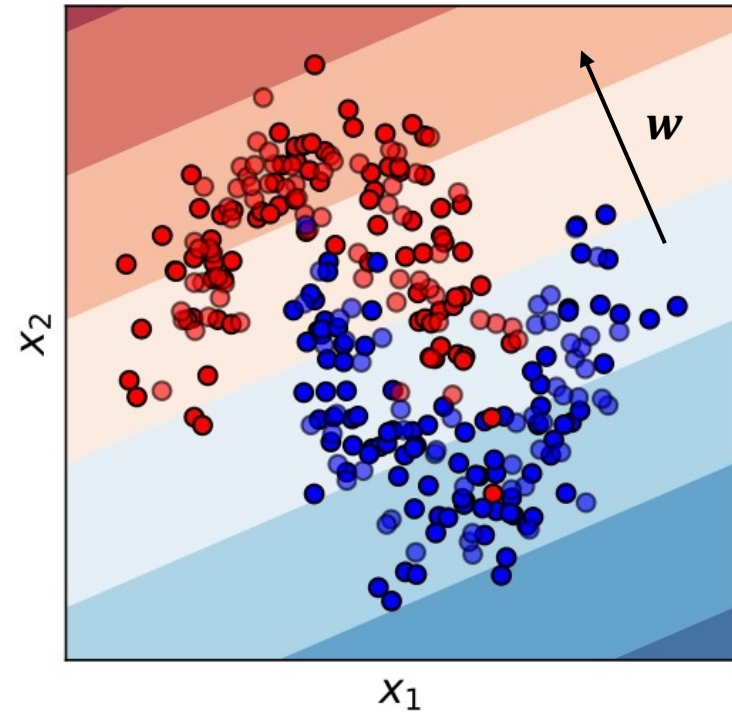
$$p(y = 1|\mathbf{x}) = \sigma\left(\sum_{j=1}^m w_j x_j + b\right)$$

$$p(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^T \mathbf{x} + b)$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

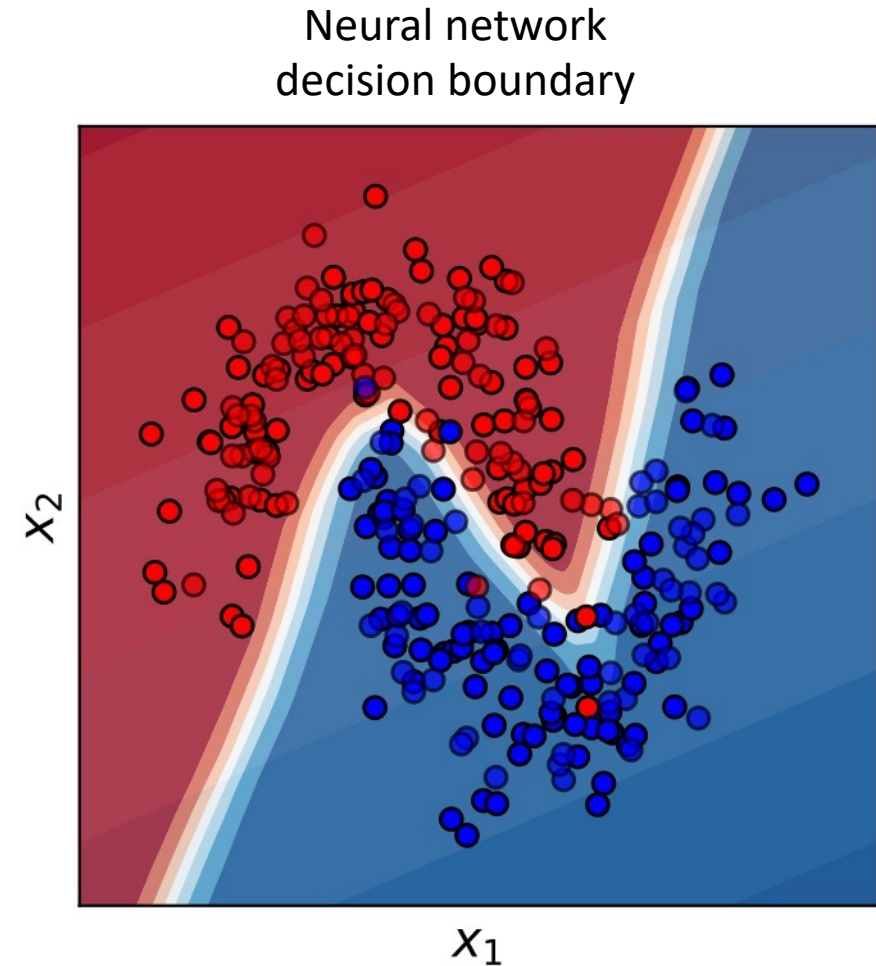
Recall: logistic regression yields linear decision boundaries.

- Predicted log-odds increase as features move in the direction defined by $\mathbf{w}$
(consider the dot-product $\mathbf{w}^T \mathbf{x}$)
- Intercept b allows predicted log-odds to be non-zero at the origin
- Linear decision boundaries aren't always good enough



Recall: sometimes linear decision boundaries aren't good enough.

- Suppose x_1 and x_2 are biomarker values
- After biopsy:
 - Blue patients: benign
 - Red patients: malignant
- Why can't logistic regression yield this decision boundary?
- Consider: how does increasing x_1 change the predicted log-odds?



Let's break logistic regression.

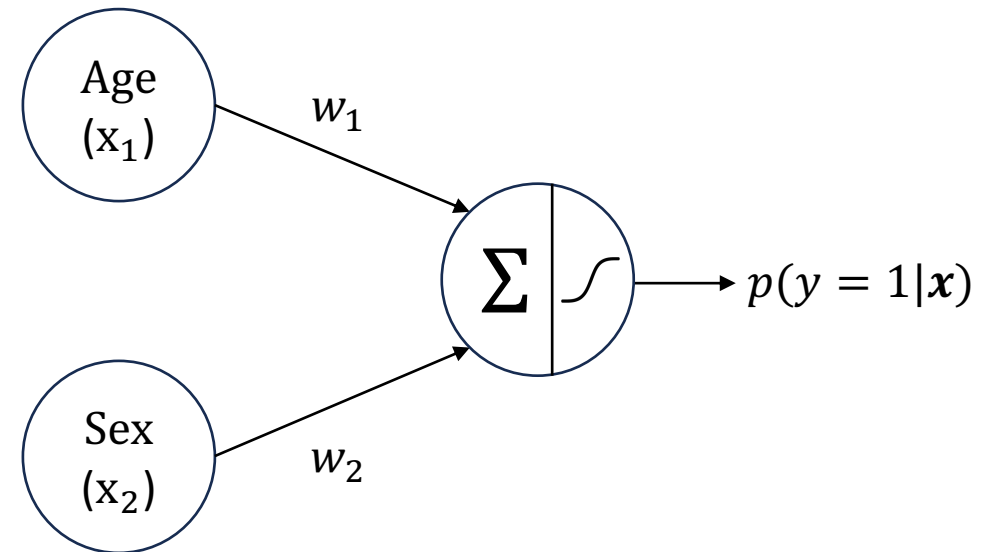
Specific examples where linear decision boundaries are insufficient

Breaking logistic regression: simple example 1

- Suppose there's a new disease. We're trying to develop a predictive model to determine who it will affect.
- For whatever reason, it turns out the disease affects only two groups of people:
 - Younger Females (under 60)
 - Older Males (over 60)
- Can logistic regression learn to predict this?

Breaking logistic regression: simple example 1

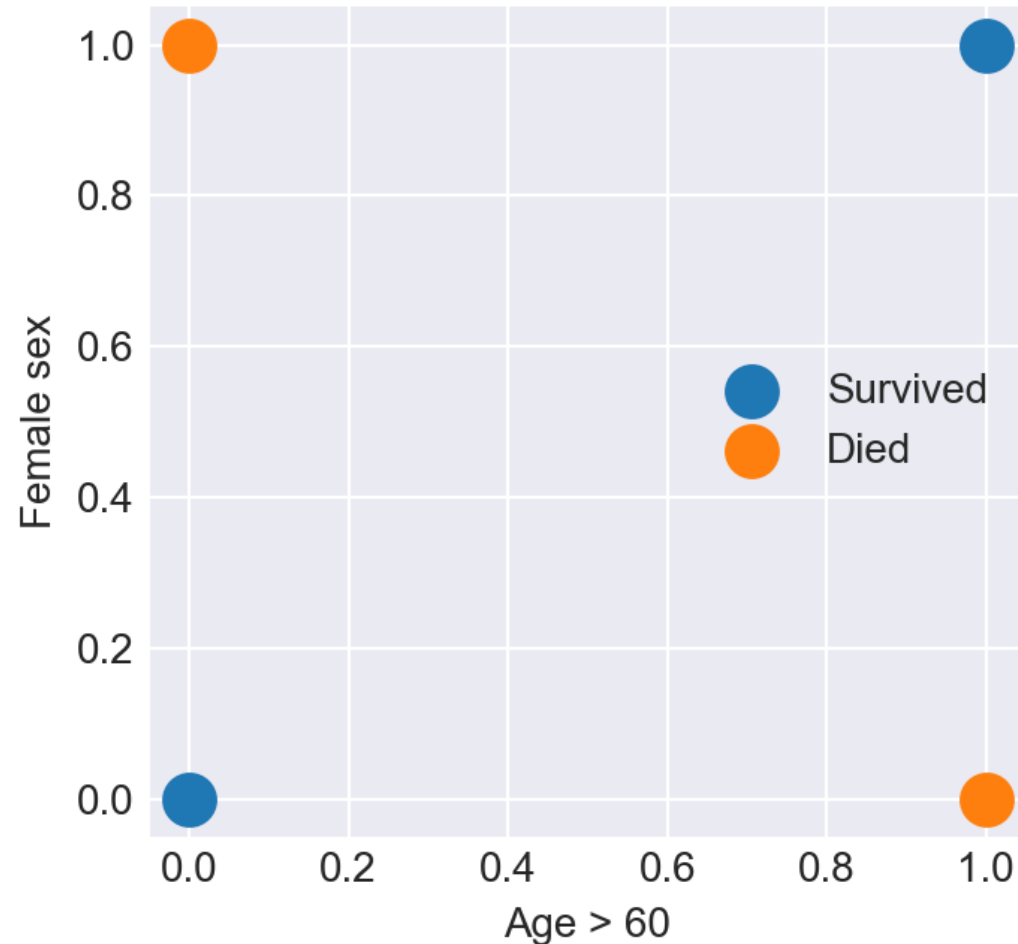
- Predictor 1: Age
 - $x_1 = 1$ if age > 60
 - $x_1 = 0$ if age ≤ 60
- Predictor 2: Sex
 - $x_2 = 1$ if female
 - $x_2 = 0$ if male
- Goal: predict high log-odds only for
 - Females under 60
 - Males over 60
- What should the parameters (b_1 and b_2) be?



Breaking logistic regression: simple example 1

Can we solve this with a linear decision boundary?

In other words, **can we draw a line separating those who lived from those who died?**

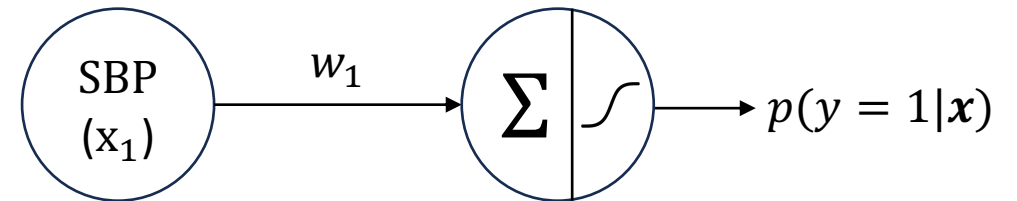


Breaking logistic regression: simple example 2

- In the ED, we're trying to develop a model that predicts mortality from systolic blood pressure upon arrival.
- It turns out, you're at high risk of dying if you:
 - a) You have very high blood pressure, OR
 - b) You have very low blood pressure
- Can logistic regression learn to predict this?

Breaking logistic regression: simple example 2

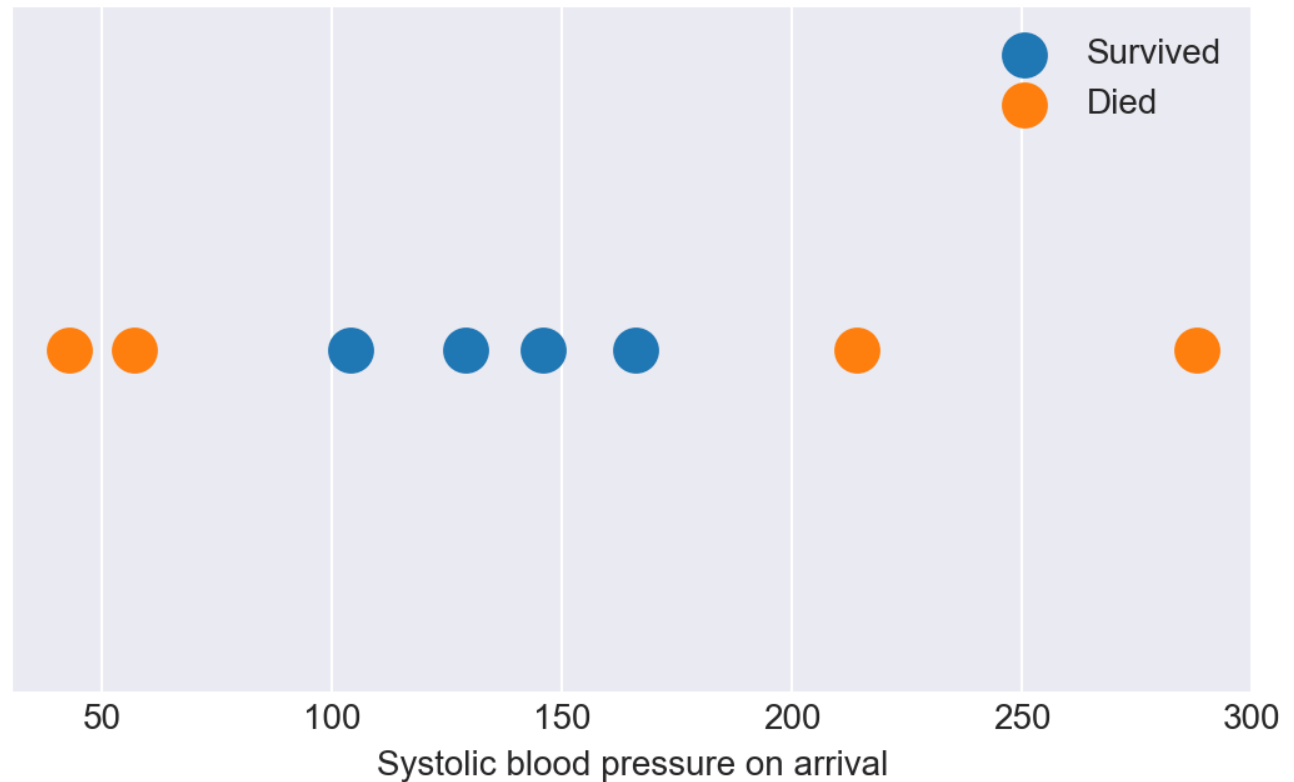
- Predictor 1: Systolic blood pressure
- Goal: predict high log-odds only for
 - SBP > 200
 - SBP < 60
- What should the parameter (b_1) be?



Breaking logistic regression: simple example 2

Can we solve this with a linear decision boundary?

In other words, **can we draw a line separating those who lived from those who died?**

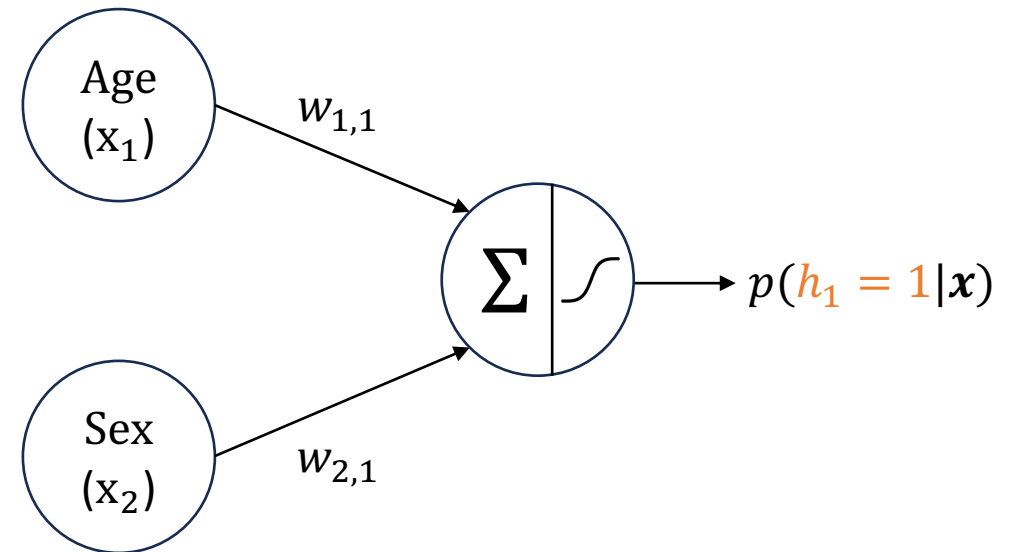


In general, there can be:

- Nonlinear effects (e.g. *high* and *low* blood pressure both increase risk; middle/normal blood pressure is OK)
- Interactions: males over 60 are at risk, and females under 60 are at risk, but being male (or female) does not on its own increase risk
- *Subgroups* we'd like to identify that may be at risk
- We need models that learn these relationships rather than relying on predefined features
- We'll use logistic regression as a starting point to build more flexible models

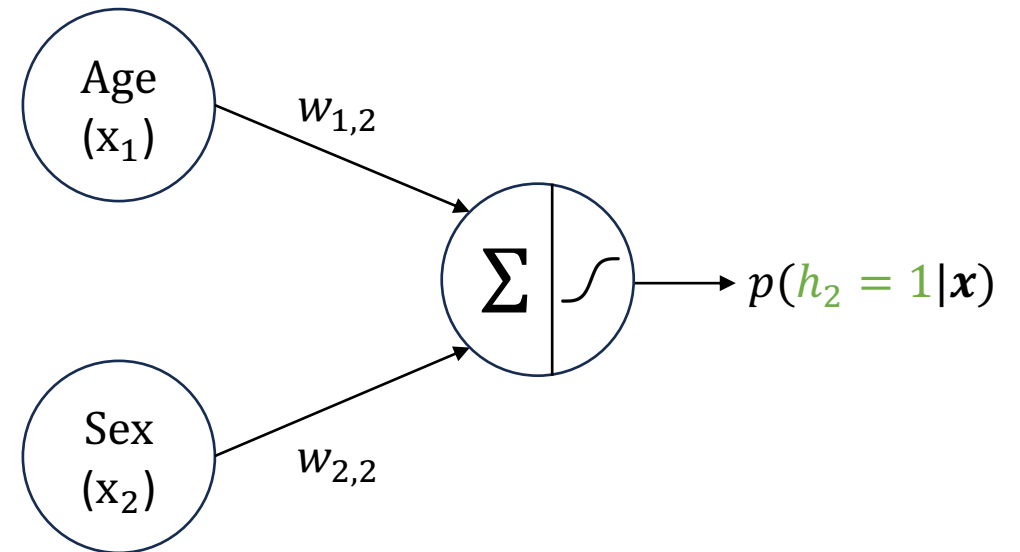
Let's break the problem into simpler pieces.

- Predictor 1: Age
 - $x_1 = 1$ if age > 60
 - $x_1 = 0$ if age ≤ 60
- Predictor 2: Sex
 - $x_2 = 1$ if female
 - $x_2 = 0$ if male
- Can logistic regression identify
 - Females under 60 ($h_1=1$)?
 - Males over 60 ($h_2=1$)?
- What should the parameters be?



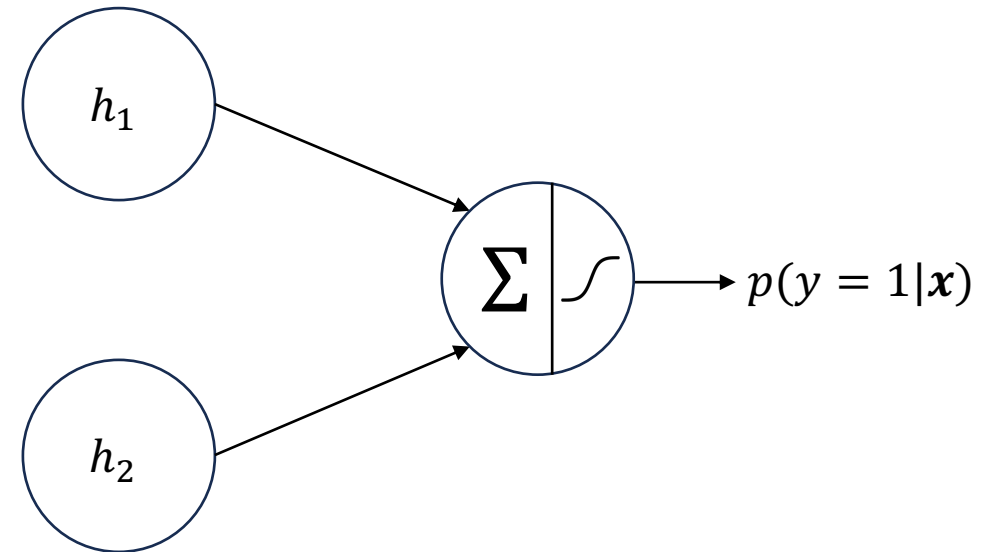
Let's break the problem into simpler pieces.

- Predictor 1: Age
 - $x_1 = 1$ if age > 60
 - $x_1 = 0$ if age ≤ 60
- Predictor 2: Sex
 - $x_2 = 1$ if female
 - $x_2 = 0$ if male
- Can logistic regression identify
 - Females under 60 ($h_1=1$)?
 - Males over 60 ($h_2=1$)?
- What should the parameters be?



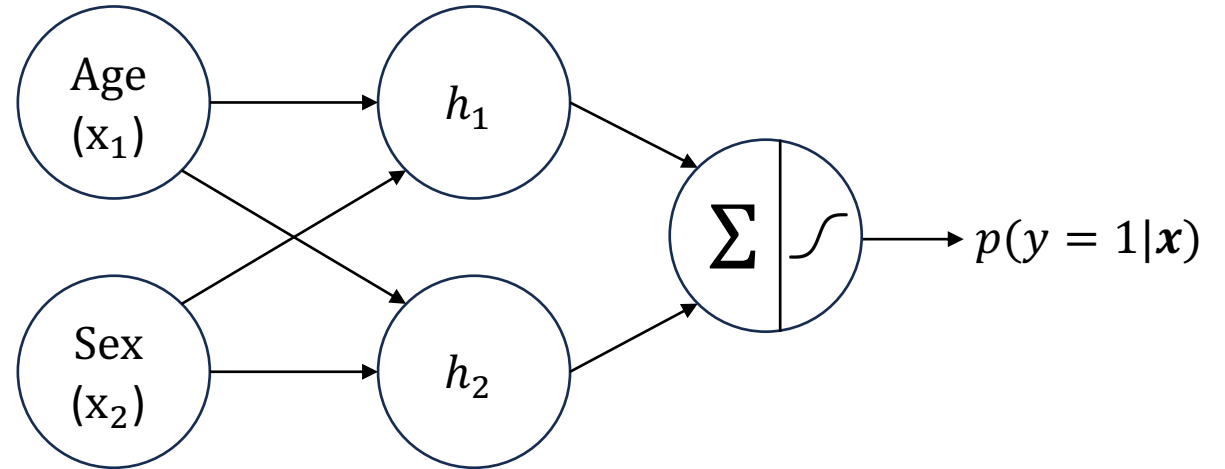
Now, *use these predictions*
to make predictions

- Neuron 1 (h_1):
 - $h_1 = 1$ if (female ≤ 60)
 - $h_1 = 0$ otherwise
- Neuron 2 (h_2):
 - $h_2 = 1$ if (male > 60)
 - $h_2 = 0$ otherwise
- What should the parameters be?



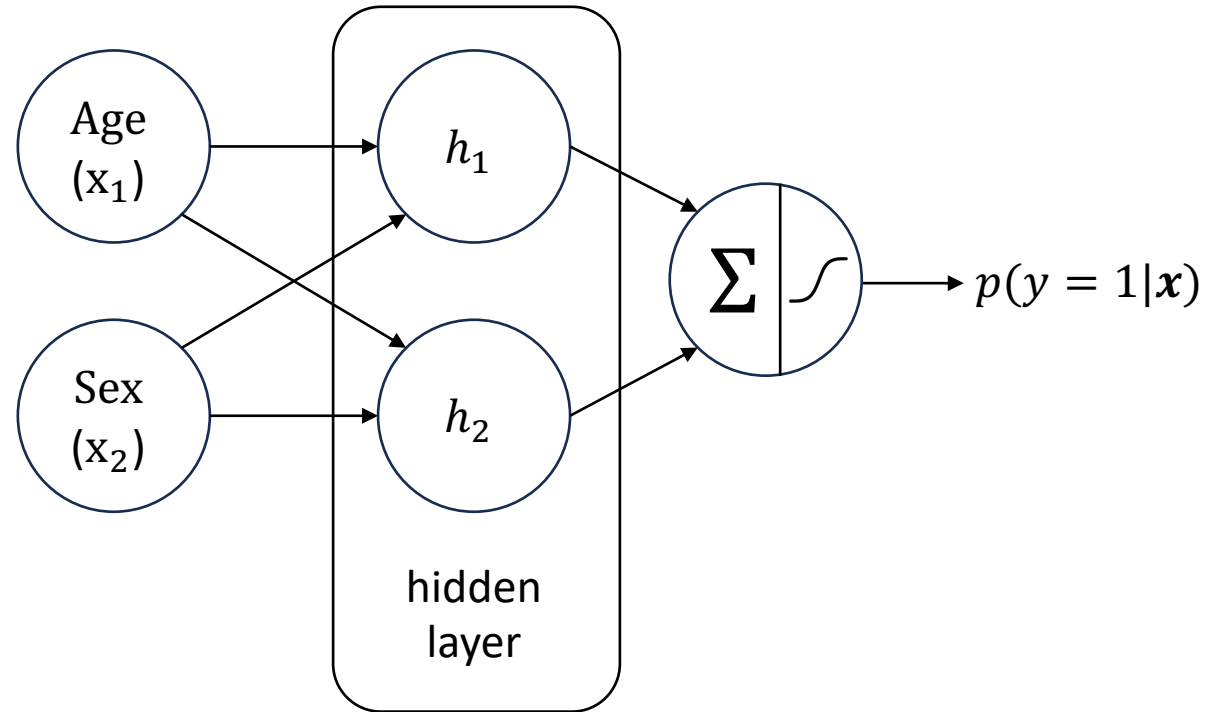
This is a (very simple) fully-connected neural network.

- Neuron 1 (h_1):
 - $h_1 = 1$ if (female ≤ 60)
 - $h_1 = 0$ otherwise
- Neuron 2 (h_2):
 - $h_2 = 1$ if (male > 60)
 - $h_2 = 0$ otherwise
- What should the parameters be?



This is a (very simple) fully-connected neural network.

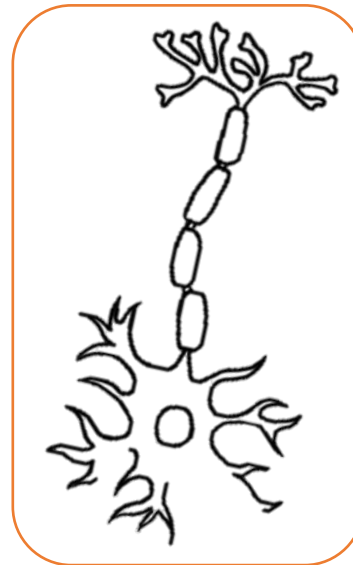
- Neuron 1 (h_1):
 - $h_1 = 1$ if (female ≤ 60)
 - $h_1 = 0$ otherwise
- Neuron 2 (h_2):
 - $h_2 = 1$ if (male > 60)
 - $h_2 = 0$ otherwise
- What should the parameters be?



This is a (very simple) fully-connected neural network.

- Each logistic regression is like a neuron

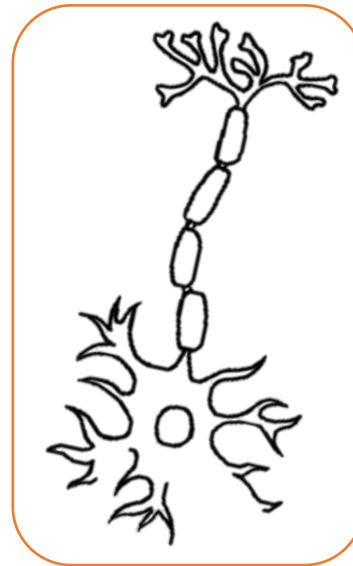
Neuron 1 (h_1):
Detects females under 60



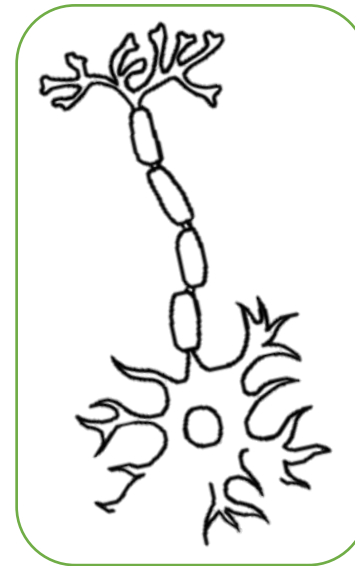
This is a (very simple) fully-connected neural network.

- Each logistic regression is like a neuron
- Different neurons detect different *features*
 - *Feature 1*: female under 60
 - *Feature 2*: male over 60

Neuron 1 (h_1):
Detects females under 60



Neuron 2 (h_2):
Detects males over 60



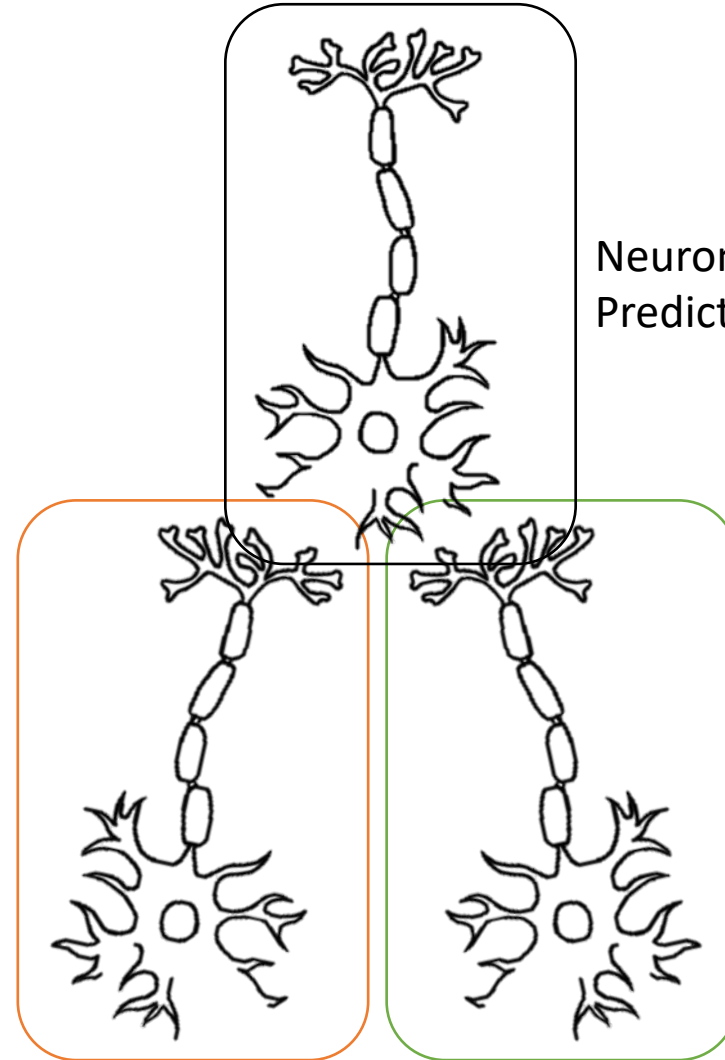
This is a (very simple) fully-connected neural network.

- Each logistic regression is like a neuron
- Different neurons detect different *features*
 - *Feature 1*: female under 60
 - *Feature 2*: male over 60
- Predictions are made based on detected features rather than the original predictors

Neuron 1 (h_1):
Detects females under 60

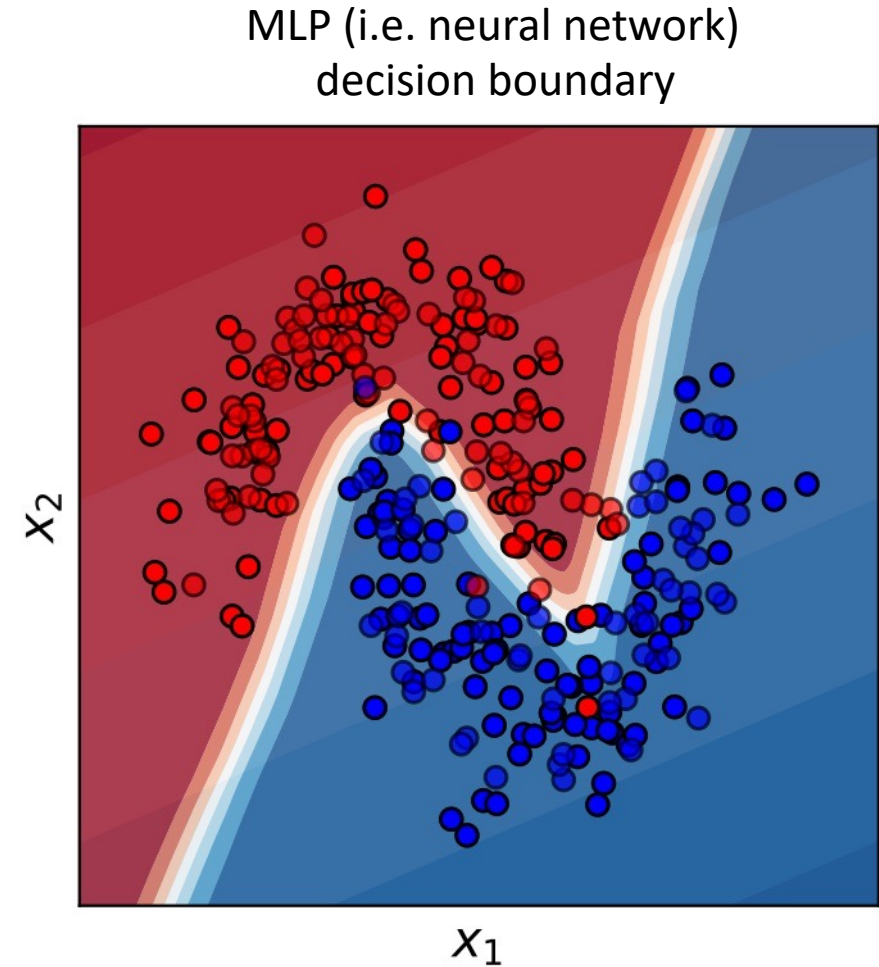
Neuron 3:
Predicts the outcome

Neuron 2 (h_2):
Detects males over 60

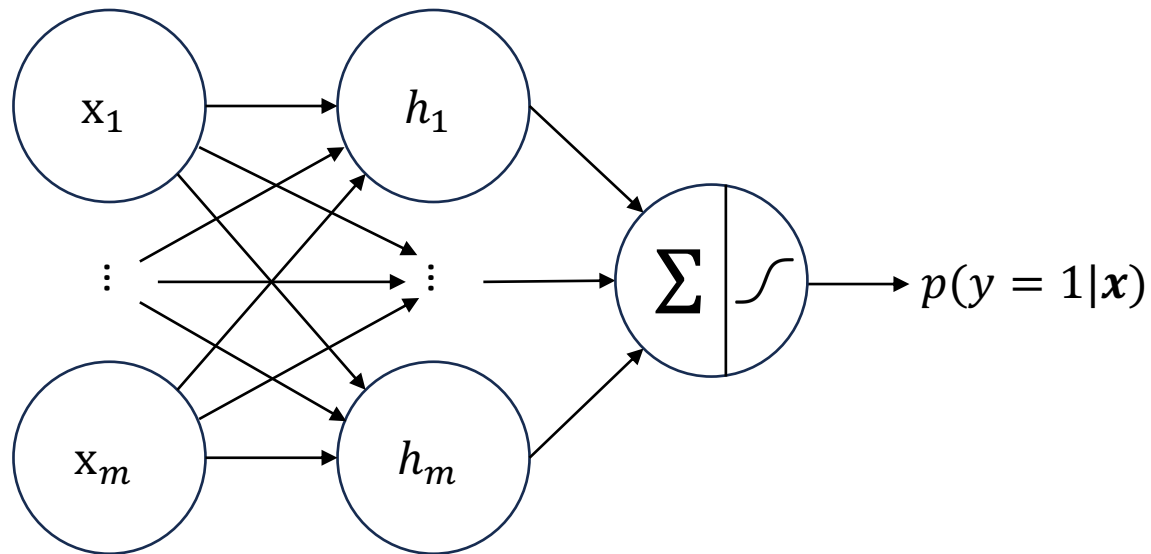


Neural networks provide unlimited flexibility.

- Add flexibility:
 - More layers of feature detectors
 - More feature detectors per layer



A shallow, wide neural network

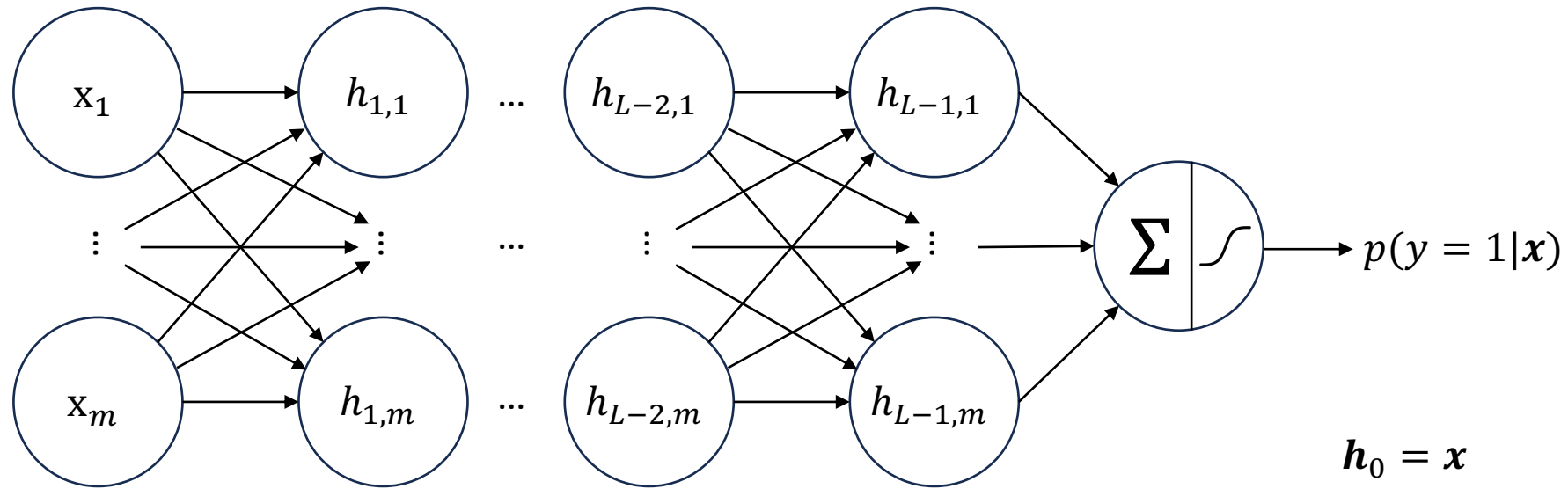


Note that $\mathbf{h}$ is
“fully connected” to $\mathbf{x}$

$$\mathbf{h} = \sigma(W_1\mathbf{x} + \mathbf{b}_1)$$

$$p(y = 1|\mathbf{x}) = \sigma(W_L\mathbf{h} + b_L)$$

A deep neural network



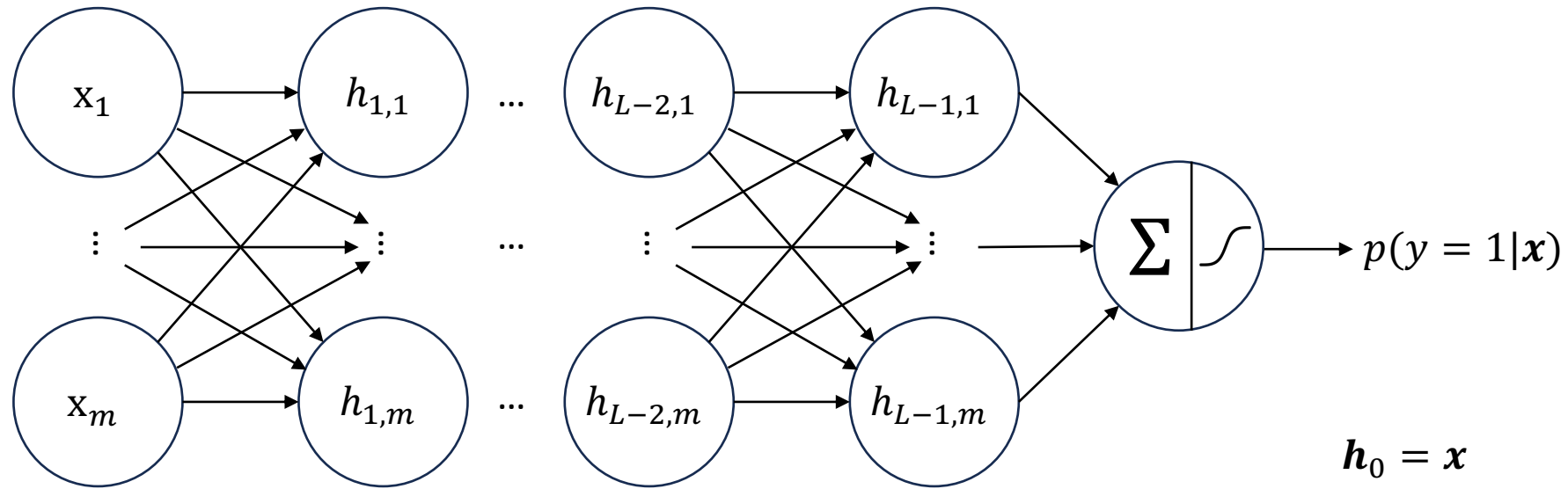
Note that $\mathbf{h}$ is
“fully connected” to $\mathbf{x}$

$$\mathbf{h}_0 = \mathbf{x}$$

$$\mathbf{h}_l = \sigma(W_l \mathbf{h}_{l-1} + \mathbf{b}_l)$$

$$p(y = 1|\mathbf{x}) = \sigma(W_L \mathbf{h}_{L-1} + b_L)$$

A deep neural network



Note that $\mathbf{h}$ is
“fully connected” to $\mathbf{x}$

$$\mathbf{h}_0 = \mathbf{x}$$

$$\mathbf{h}_l = \sigma(W_l \mathbf{h}_{l-1} + \mathbf{b}_l)$$

$$p(y = 1 | \mathbf{x}) = \sigma(W_L \mathbf{h}_{L-1} + b_L)$$

Note: layers can have different numbers
of *hidden units* (i.e. neurons)

Report back: counting parameters

How do the number of NN parameters scale with:

- a) the width of each hidden layer?
- b) the number of hidden layers?

(discuss and report back in a few minutes)

Neural networks are universal function approximators

Let $u(t)$ denote the unit step function:

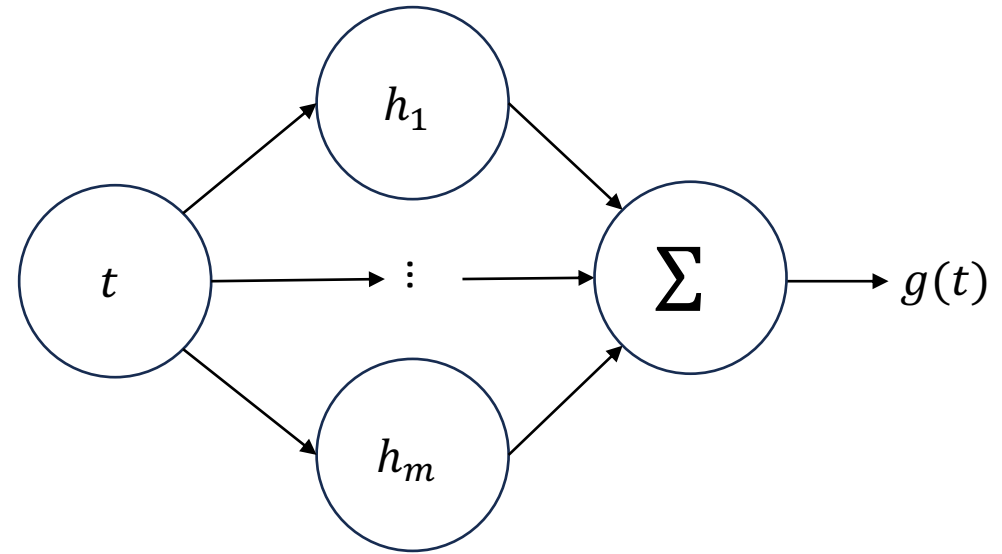
$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

Note that $u(t) = \lim_{\alpha \rightarrow \infty} \sigma(\alpha t)$ for all $t \neq 0$.

We'd like to construct a neural network g that approximates a smooth function f on $[0, T]$.

Our network has a single hidden layer of size m , where each h_m is defined as follows:

$$h_j(t) = u(t - t_j) = \begin{cases} 1, & t \geq t_j \\ 0, & t < t_j \end{cases} \text{ where } t_j = \frac{j \cdot T}{m}$$



Neural networks are universal function approximators

Let $u(t)$ denote the unit step function:

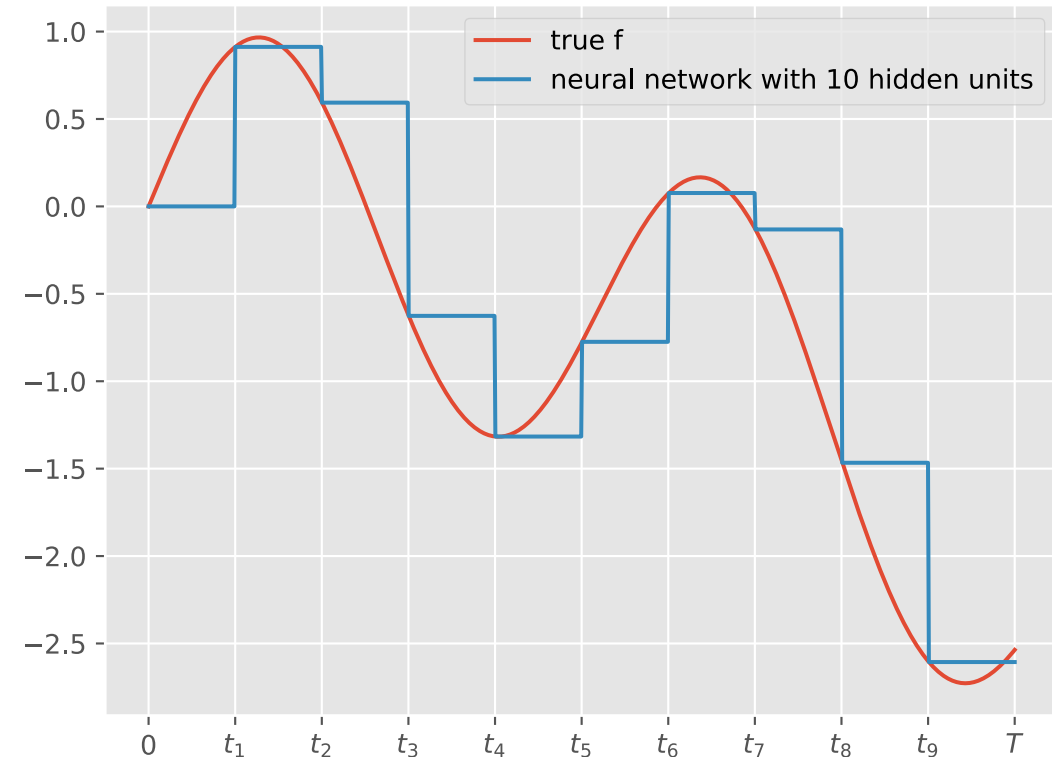
$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

Note that $u(t) = \lim_{\alpha \rightarrow \infty} \sigma(\alpha t)$ for all $t \neq 0$.

We'd like to construct a neural network g that approximates a smooth function f on $[0, T]$.

Our network has a single hidden layer of size m , where each h_m is defined as follows:

$$h_j(t) = u(t - t_j) = \begin{cases} 1, & t \geq t_j \\ 0, & t < t_j \end{cases} \text{ where } t_j = \frac{j \cdot T}{m}$$



Consider: what are the associated weights w_j ?

Neural networks are universal function approximators

Let $u(t)$ denote the unit step function:

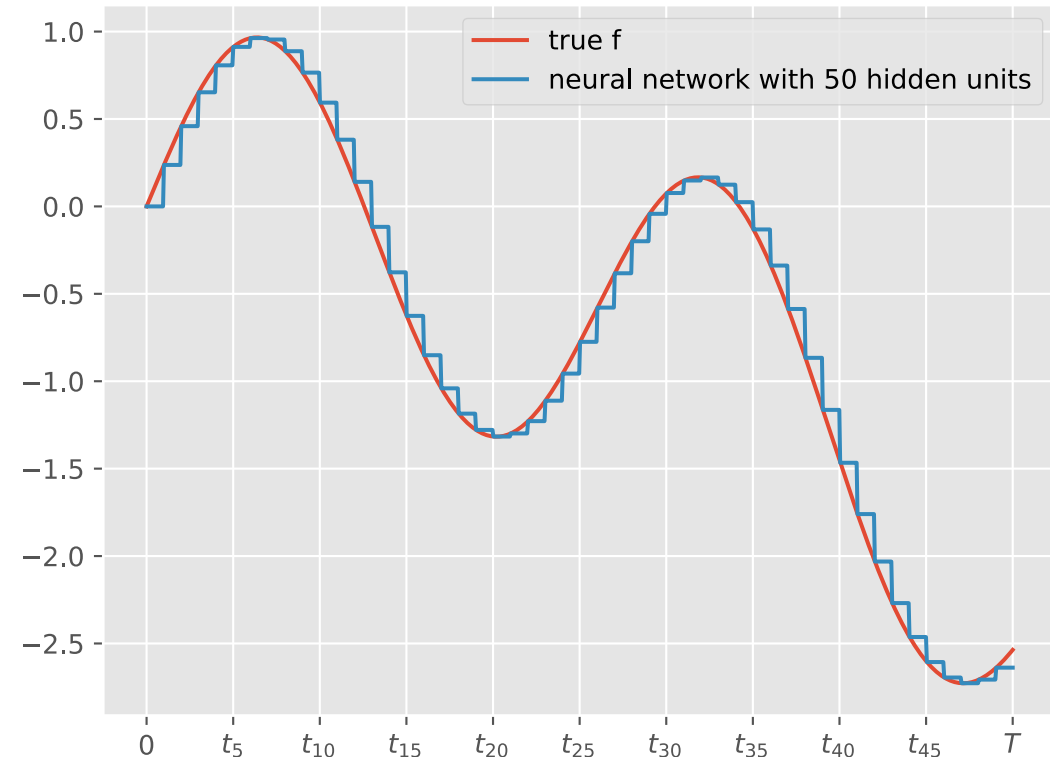
$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

Note that $u(t) = \lim_{\alpha \rightarrow \infty} \sigma(\alpha t)$ for all $t \neq 0$.

We'd like to construct a neural network g that approximates a smooth function f on $[0, T]$.

Our network has a single hidden layer of size m , where each h_m is defined as follows:

$$h_j(t) = u(t - t_j) = \begin{cases} 1, & t \geq t_j \\ 0, & t < t_j \end{cases} \text{ where } t_j = \frac{j \cdot T}{m}$$



Consider: what are the associated weights w_j ?

Neural networks are universal function approximators

Let $u(t)$ denote the unit step function:

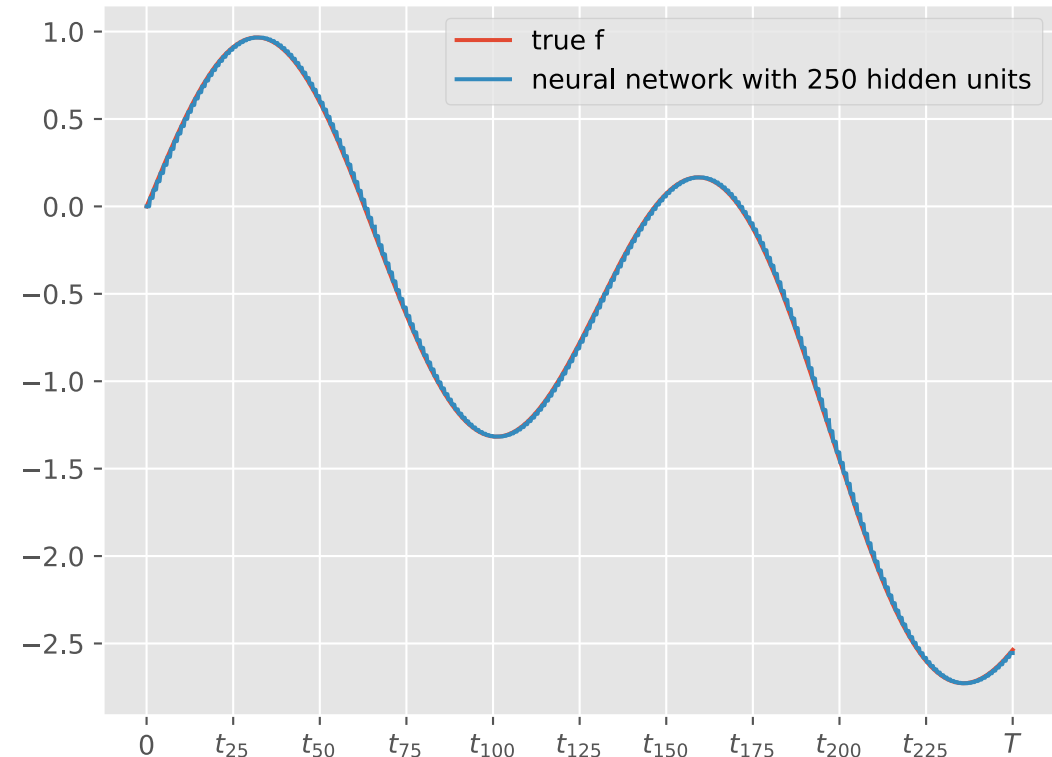
$$u(t) = \begin{cases} 1, & t \geq 0 \\ 0, & t < 0 \end{cases}$$

Note that $u(t) = \lim_{\alpha \rightarrow \infty} \sigma(\alpha t)$ for all $t \neq 0$.

We'd like to construct a neural network g that approximates a smooth function f on $[0, T]$.

Our network has a single hidden layer of size m , where each h_m is defined as follows:

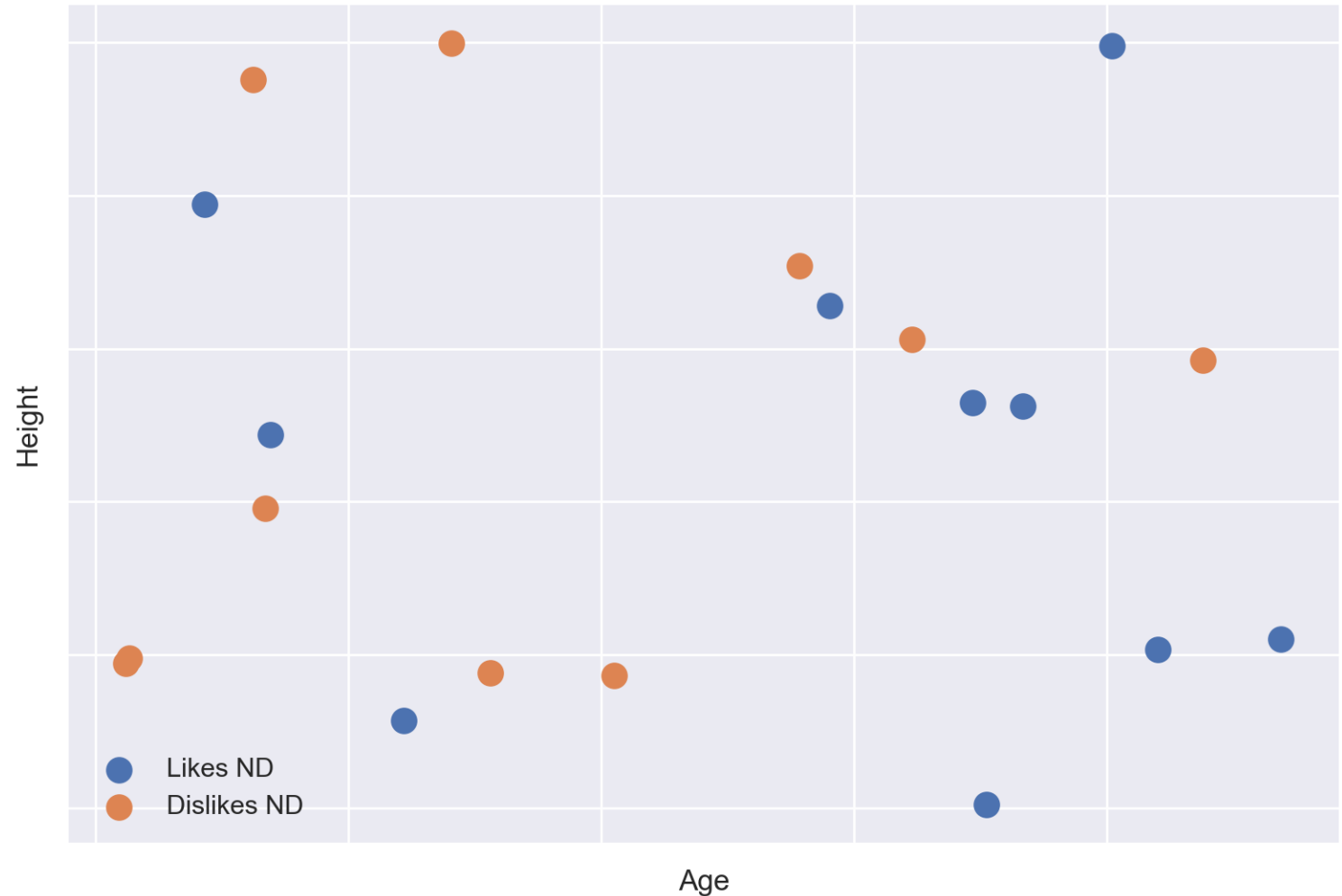
$$h_j(t) = u(t - t_j) = \begin{cases} 1, & t \geq t_j \\ 0, & t < t_j \end{cases} \text{ where } t_j = \frac{j \cdot T}{m}$$



Consider: what are the associated weights w_j ?

Neural networks are perfect classifiers

Question: Can logistic regression classify all points correctly?

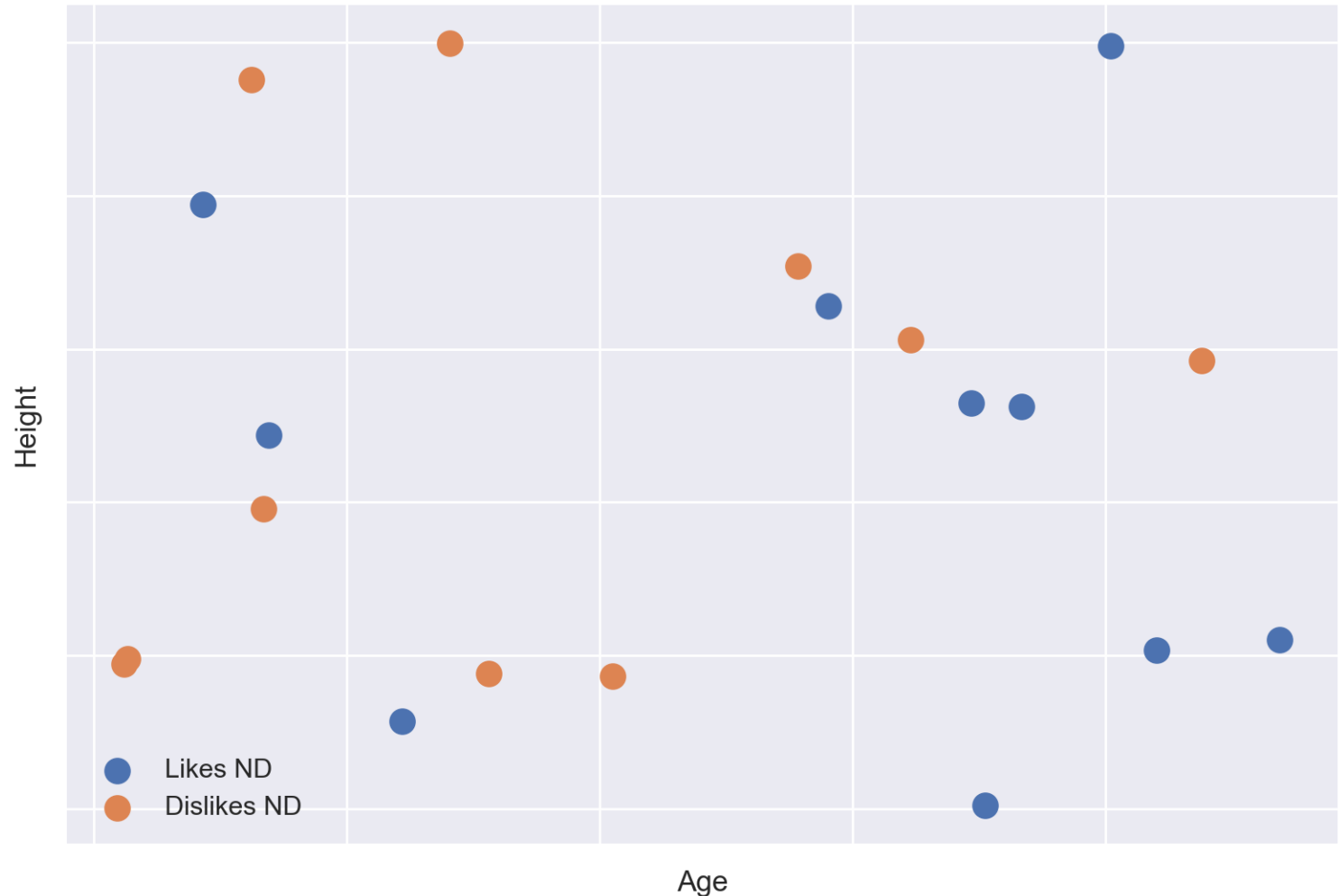


Neural networks are perfect classifiers

Question: Can logistic regression classify all points correctly?

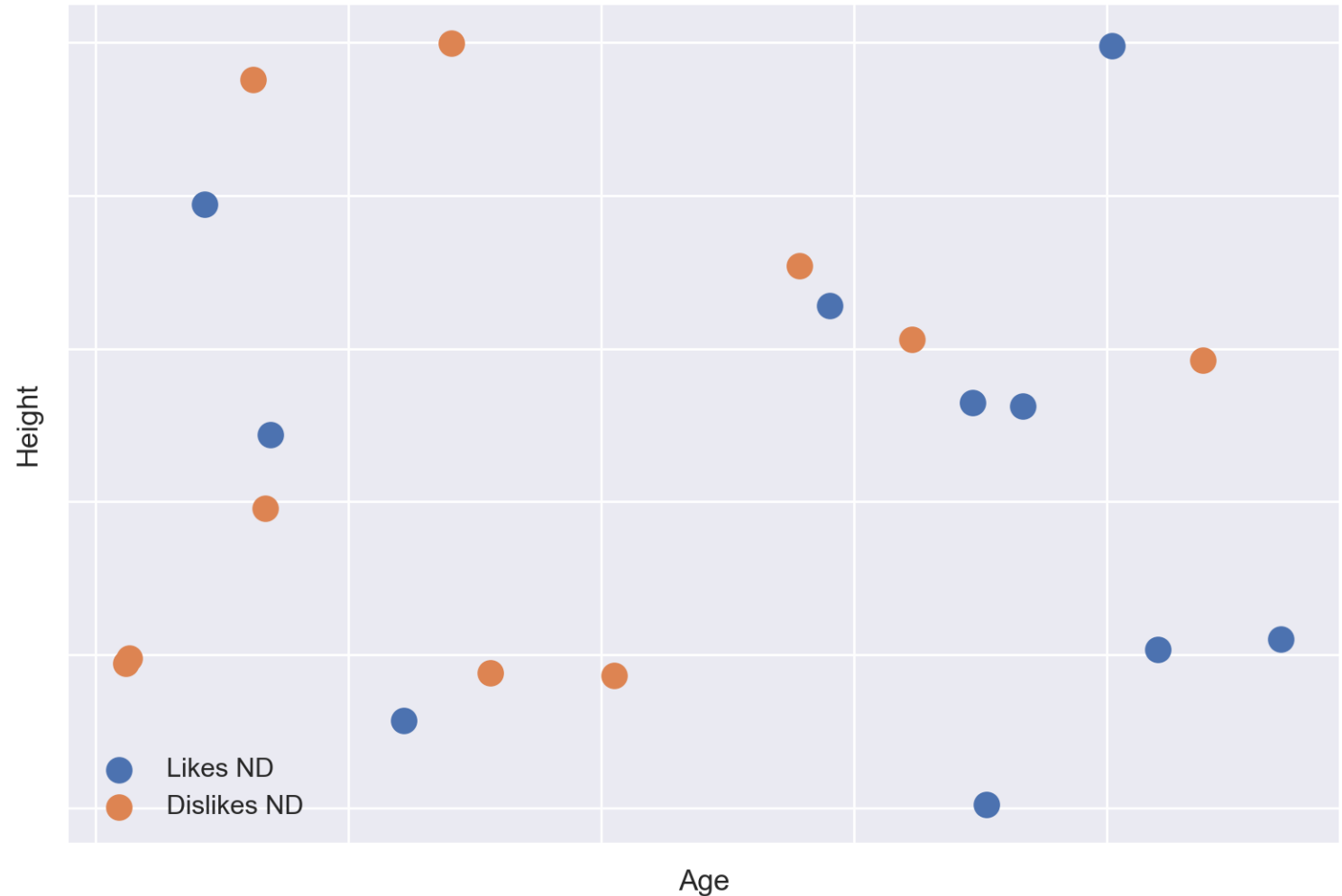
Answer: No. There is no line that perfectly separates the blue and orange points.

*note that here we have
 $\text{num_points} \gg \text{num_features}$



Neural networks are perfect classifiers

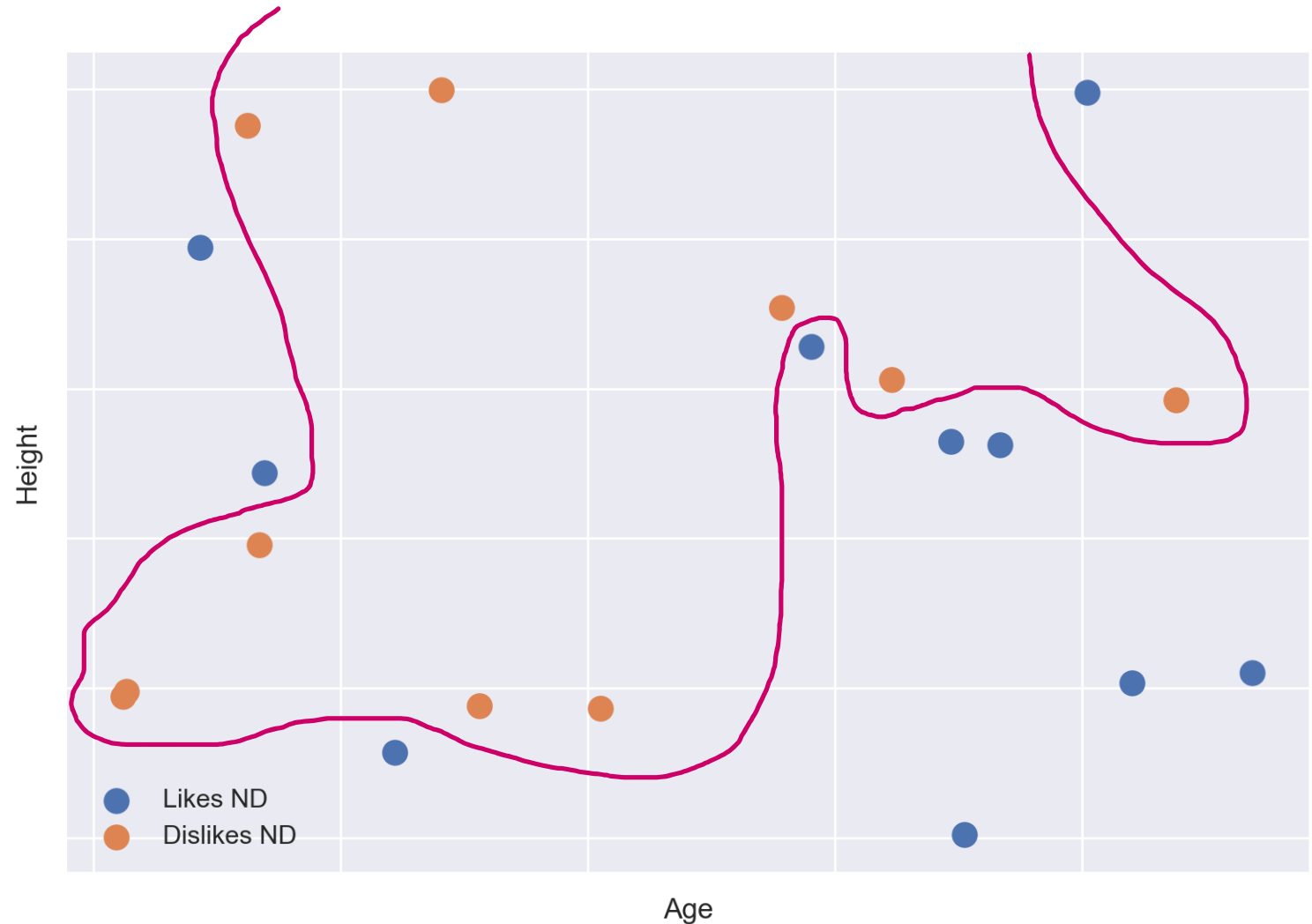
Question: Can a neural network classify all points correctly?



Neural networks are perfect classifiers

Question: Can a neural network classify all points correctly?

Answer: Yes... like so -->



Neural networks are perfect classifiers

What is the practical importance of this?

- 1) Flexible enough to solve real, difficult problems 😊

Neural networks are perfect classifiers

What is the practical importance of this?

- 1) Flexible enough to solve real, difficult problems 😊
- 2) So flexible that they can easily overfit datasets ☹️

Report back: adding an intervening layer

Consider a fully connected neural network with L hidden layers. Suppose not all layers contain the same number of hidden units, and write the number of hidden units in a given layer l as m_l .

We then modify the architecture by placing a new layer h_* containing m_* hidden units between layers l and $l + 1$.

How does the number of parameters in the network change?

(discuss and report back in a few minutes)

Q: Why depth over width?

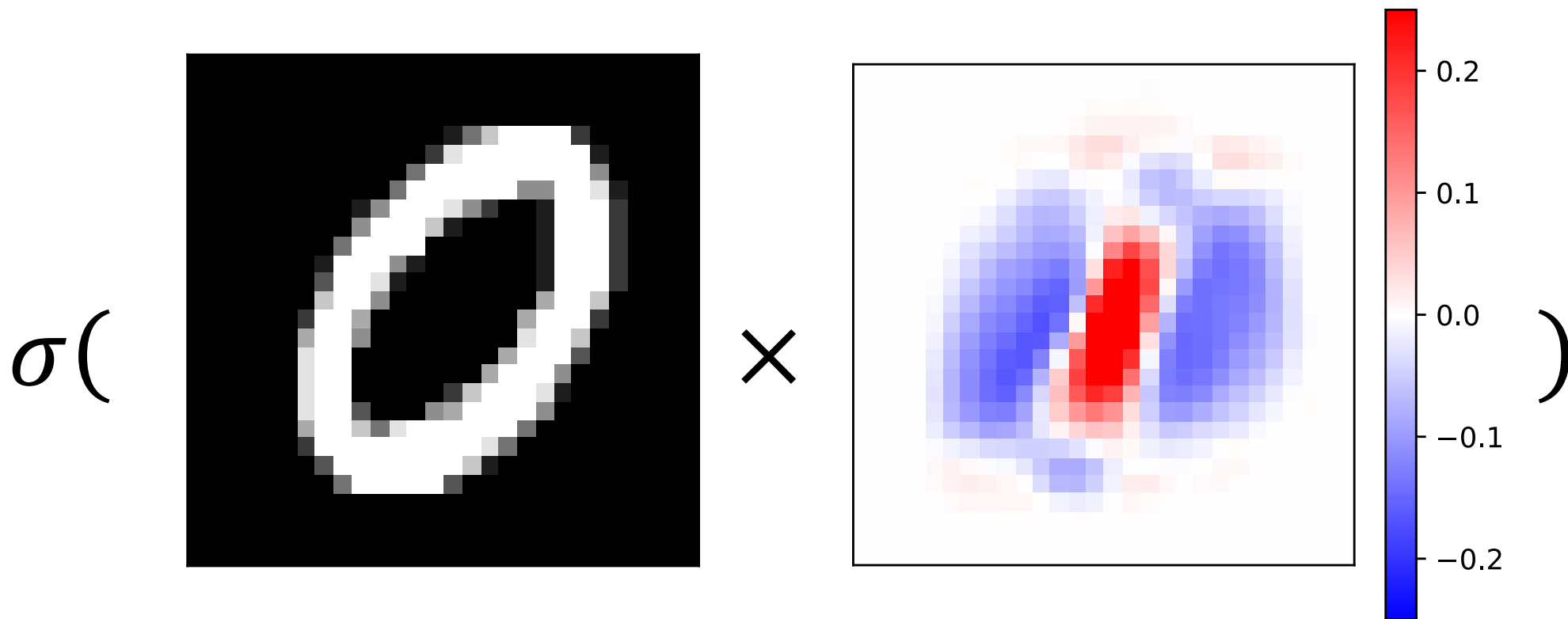
Q: Why depth over width?

A: It is more parameter-efficient!

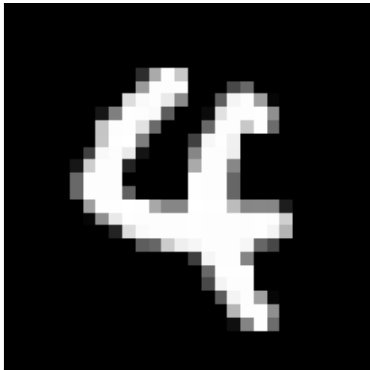
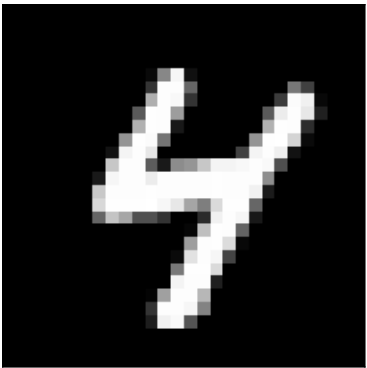
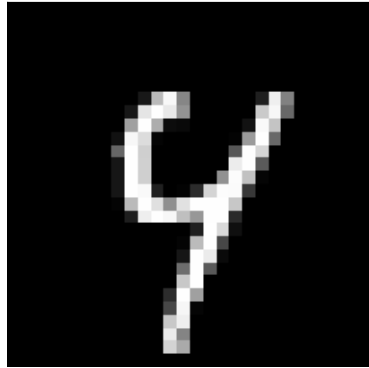
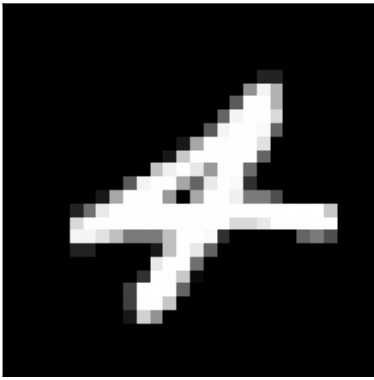
Neural network for MNIST

Moving toward computer vision

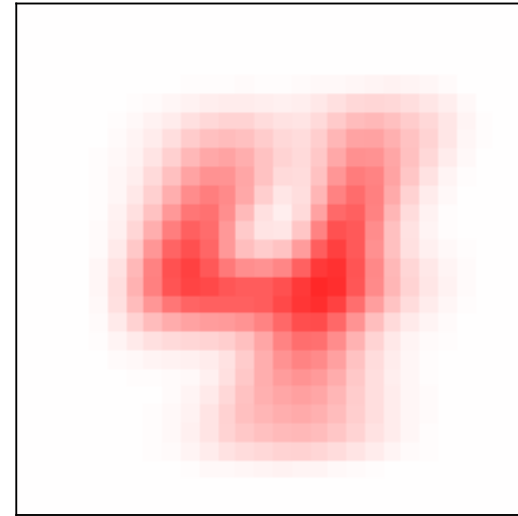
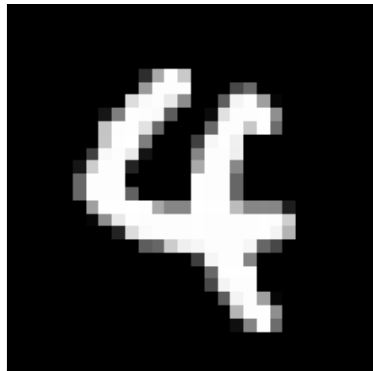
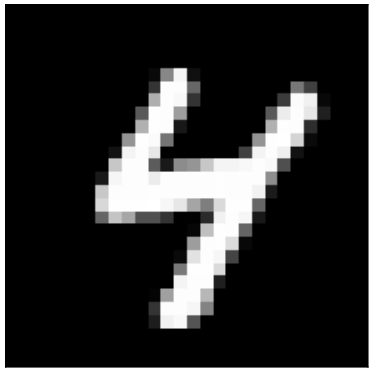
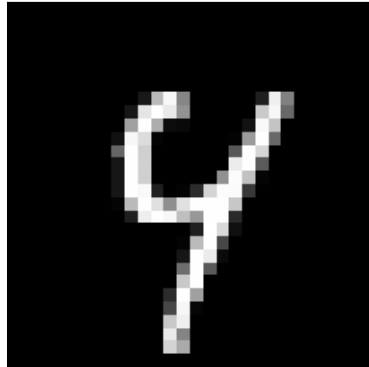
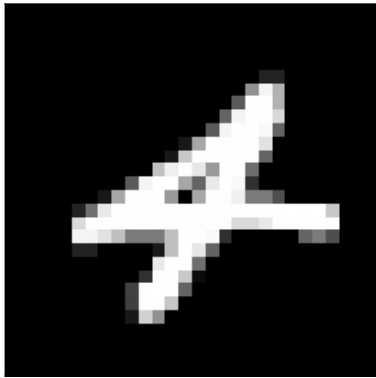
Why Limit Ourselves to Only One Filter?



Return to MNIST:
Many ways of writing “4”

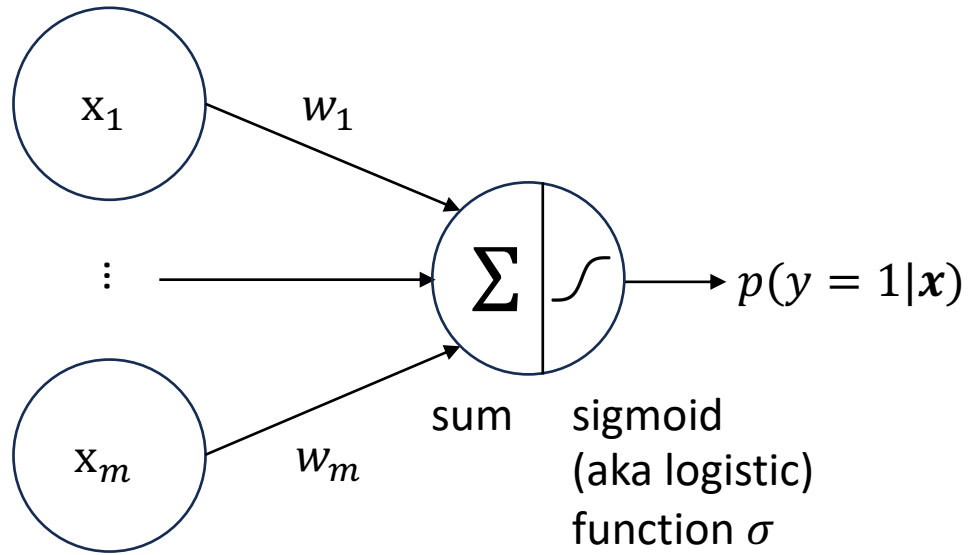


Return to MNIST: Many ways of writing “4”

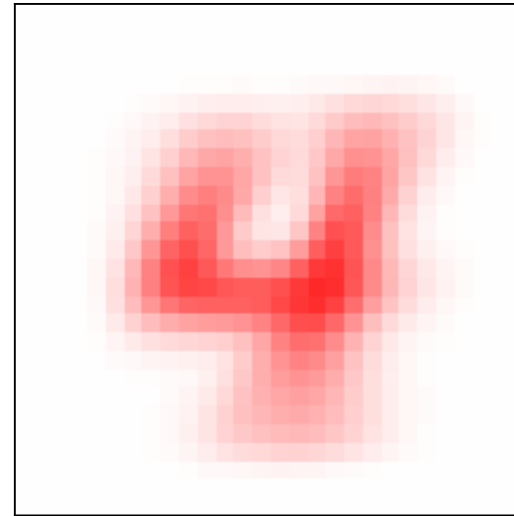


A single filter (i.e., logistic regression)
looks for the average shape

A single '4' detector

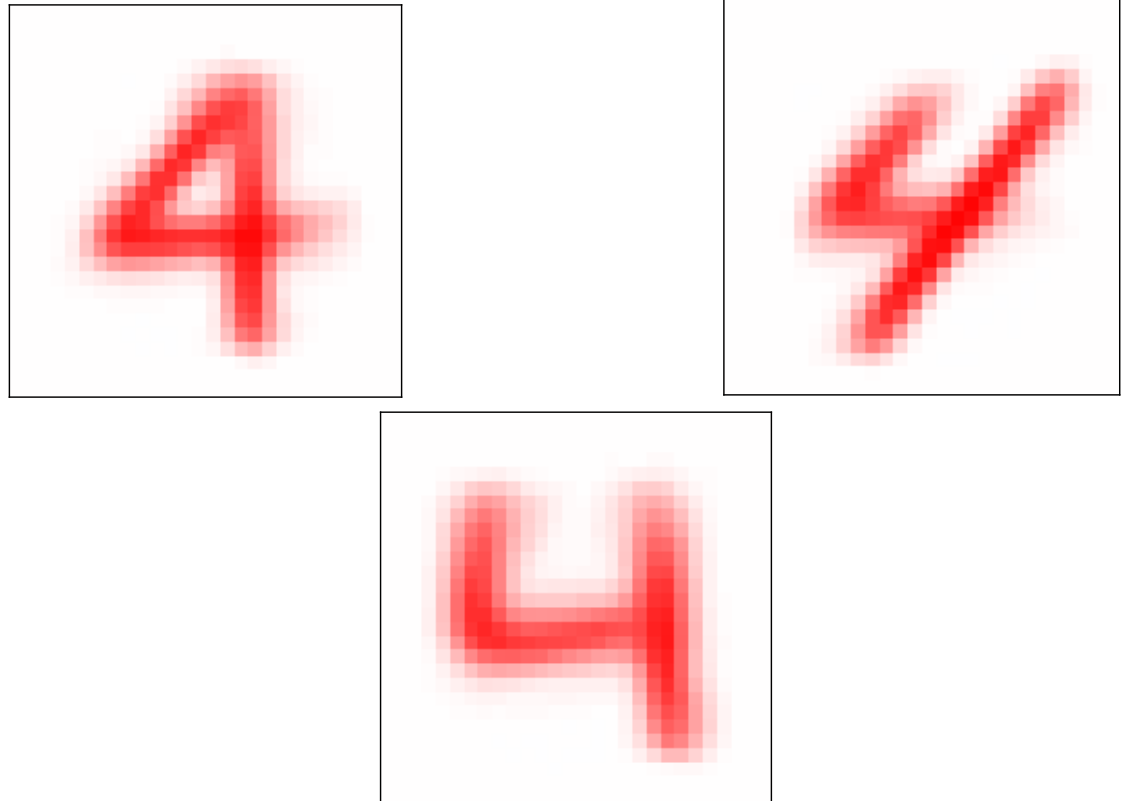


Our model detects images that look like this



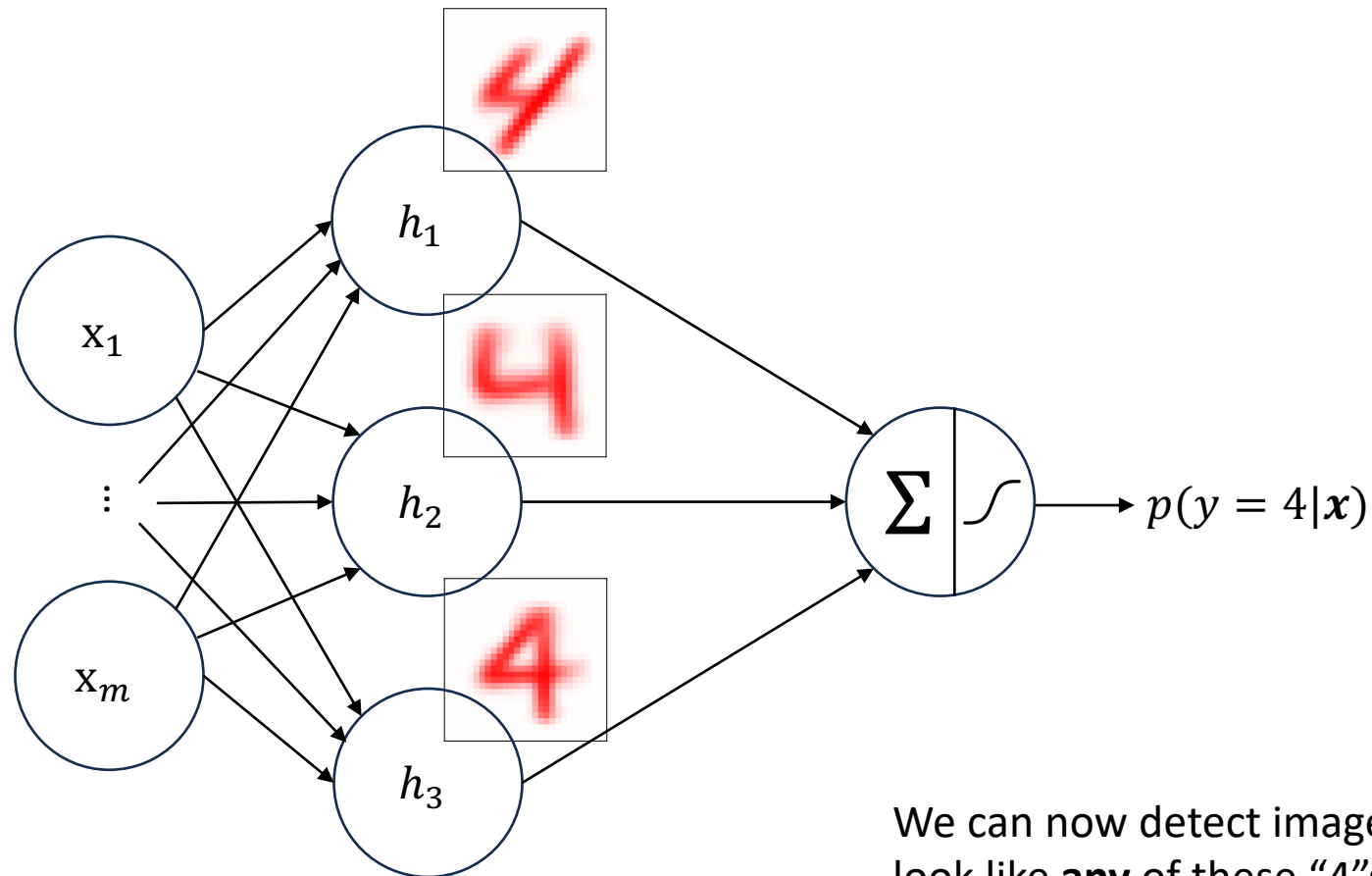
The parameters $\mathbf{w}$ after reshaping

Return to MNIST: Many ways of writing “4”



Multiple filters can look for *subtypes*
indicative of different ways of writing “4”

A more flexible '4' detector



We can now detect images that look like any of these "4"s

To increase flexibility/complexity, we can:

- Increase the width (i.e. number of hidden units) in one or more hidden layers (WIDE)
- Increase the number of hidden layers (DEEP)
- *Deep learning* refers to the latter; we are building a deep hierarchy of features

Does this work with MNIST?



Logistic regression:
~91% Accurate

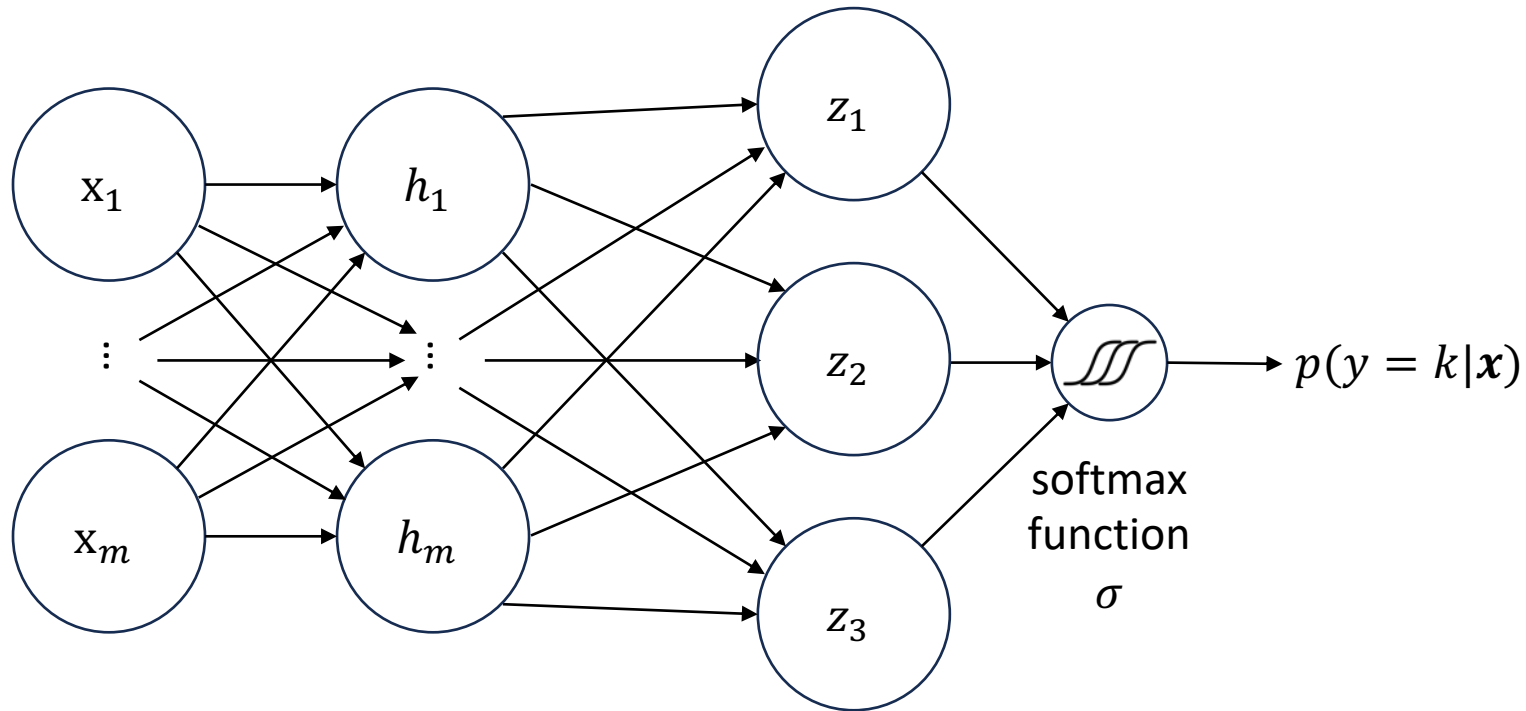
MLP with 1 hidden layer:
~96% Accurate

...maybe we can do even better...

Many different ways of being sick

- Suppose we wish to predict mortality risk from complex clinical data
- With logistic regression, our model looks for the average sick patient
- However, there are many different ways to be sick, and they may look very different from one another
- With a neural network, we can detect any of those ways of being sick, and combinations of them, much like we can detect multiple distinct ways of writing a “4”
- In principle this leads us to more effective classification

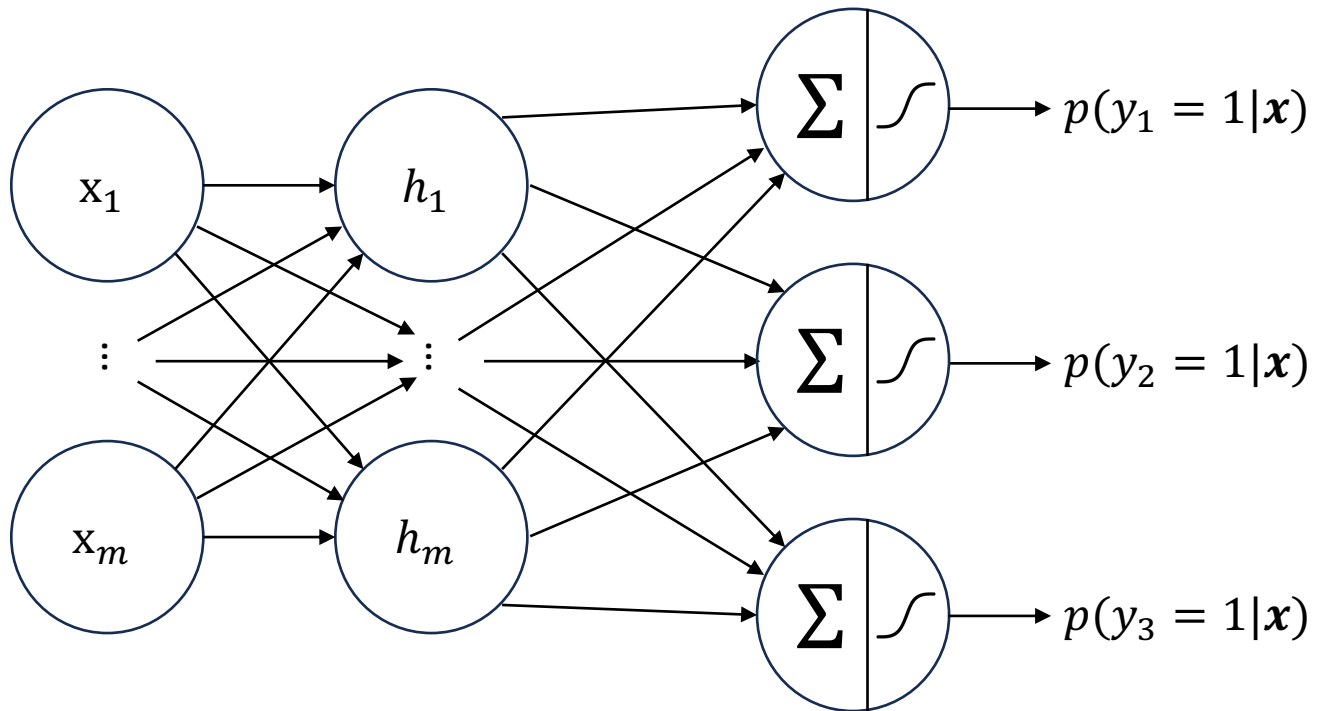
Multi-class neural networks



$$\sigma: \mathbb{R}^K \rightarrow (0, 1)^K$$

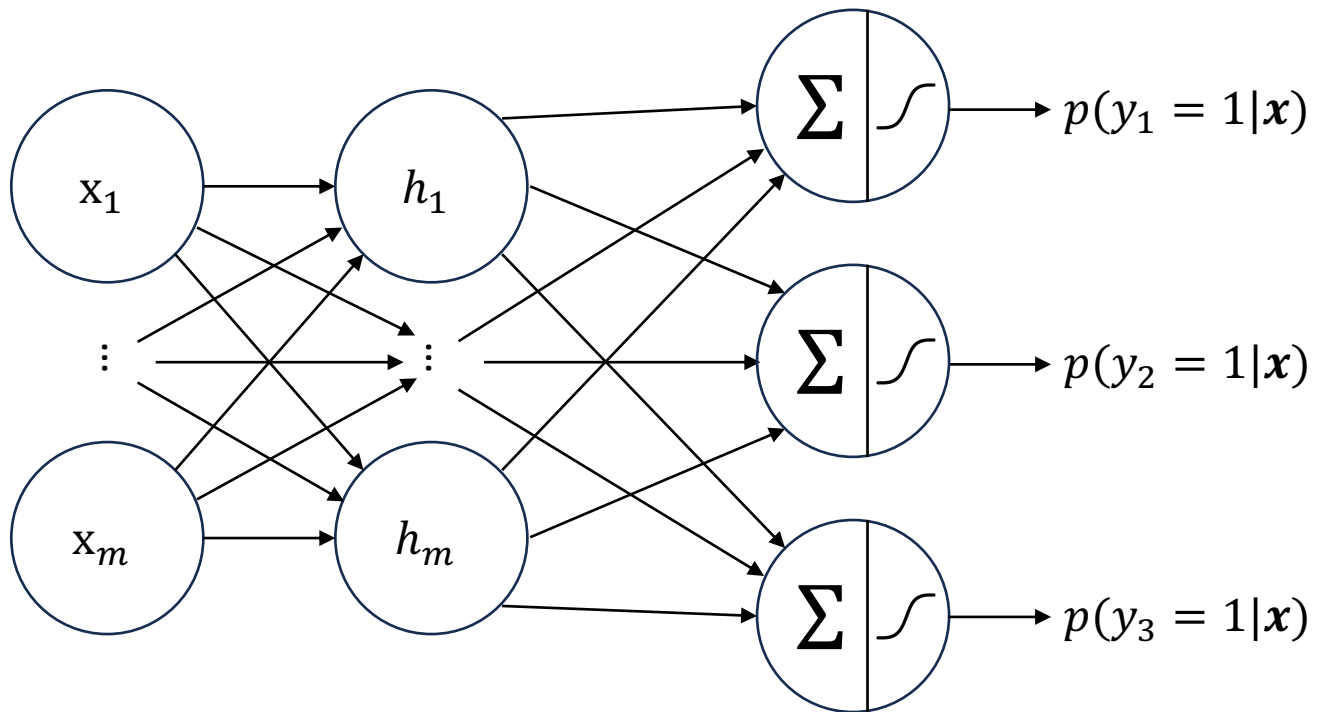
$$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

Multi-label neural networks



- Predict multiple labels from the same shared representation
- Can have a mix of label types (e.g. binary, categorical, numeric, time-to-event)
- Illustrates the flexibility of the NN/ML approach

Benefits of a multi-label approach



Report back:

Question 1: when & why might a multi-label approach be helpful?

Question 2: would it make sense to train a multi-label logistic regression model?

Summary

- There are many binary classification tasks that logistic regression can't solve.
- However, by stacking many logistic regressions together, we are able to learn intermediate, *latent* features, thereby breaking up one complex problem into many simple ones.
- This is called a multilayer perceptron (MLP), an artificial neural network (ANN), a fully connected neural network (FCNN), or just a neural network (NN).
- In NNs with multiple hidden layers, features detected by the hidden units become increasingly complex with each successive layer. We therefore say that the MLP learns a *hierarchy* of features.
- However, NNs can easily overfit data, which we will discuss more in later lectures.
- In addition to binary classification and risk prediction tasks, NNs can easily be adapted to multi-class and multi-label prediction tasks.
- The next lecture will focus on how we learn model parameters from data.